26

并发：简介

到目前为止，我们已经看到操作系统提供的几种基本抽象是怎样一步步建立起来的。我们看到了如何把一个物理 CPU 变成多个**虚拟 CPU**（virtual CPU），从而营造出多个程序同时运行的假象。我们还看到了如何为每个进程营造出一个巨大的、私有的**虚拟内存**（virtual memory）的假象：有了**地址空间**（address space）这个抽象，每个程序都可以表现得好像拥有自己的内存，而实际上是操作系统在暗中把多个地址空间复用到物理内存（有时还有磁盘）上。

本章为单个运行中的进程引入一种新的抽象：**线程**（thread）。按照经典的观点，程序中只有一个执行点（即只有一个 PC，指令从这里取出并执行）；而**多线程**（multi-threaded）程序有不止一个执行点（即有多个 PC，每个 PC 都在取指令、执行指令）。换个角度看，也可以把每个线程看成一个独立的进程，只有一点不同：线程之间共享同一个地址空间，因此可以访问同样的数据。

所以，单个线程的状态和进程的状态非常相似。线程有一个程序计数器（PC），记录程序从哪里取指令。每个线程都有一组自己私有的寄存器，用来做计算；因此，如果有两个线程在同一个处理器上运行，从运行其中一个（T1）切换到运行另一个（T2）时，就必须做一次**上下文切换**（context switch）。线程之间的上下文切换和进程之间的上下文切换很像：运行 T2 之前，必须先保存 T1 的寄存器状态，再恢复 T2 的寄存器状态。进程的状态保存在**进程控制块**（process control block，PCB）里；现在，我们需要一个或多个**线程控制块**（thread control block，TCB）来保存进程中每个线程的状态。不过，和进程间的上下文切换相比，线程间的上下文切换有一个重大区别：地址空间保持不变（也就是说，不需要切换所用的页表）。

线程和进程的另一个重大区别在于栈。在经典进程（现在可以称为**单线程**（single-threaded）进程）的简单地址空间模型中，只有一个栈，通常位于地址空间的底部（图 26.1 左）。

然而，在多线程进程中，每个线程独立运行，当然也会调用各种例程来完成手头的工作。地址空间里不再只有一个栈，而是每个线程各有一个。假设有一个多线程进程，里面有两个线程，那么它的地址空间看起来就不一样了（图 26.1 右）。

从图中可以看到，进程的地址空间里分布着两个栈。因此，所有在栈上分配的变量、参数、返回值，以及其他放在栈上的东西，都会放进有时称为**线程局部**（thread-local）存储的地方，也就是相应线程的栈里。

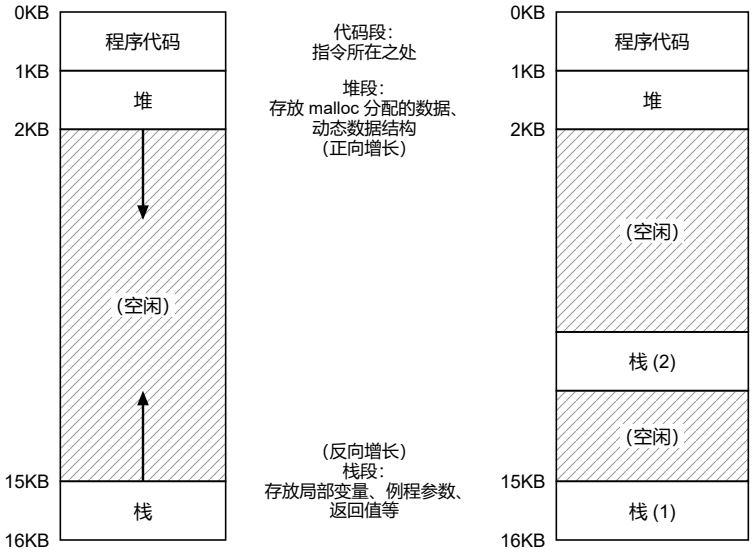


图 26.1：**单线程和多线程的地址空间**

你可能还注意到，这破坏了我们原本漂亮的地址空间布局。以前，栈和堆可以各自独立地增长，只有地址空间用完时才会出问题。现在情况就没那么好了。幸运的是，这通常不成问题，因为栈一般不需要很大（大量使用递归的程序例外）。

26.1 为什么要用线程?

在深入线程的细节、讨论编写多线程程序时可能遇到的问题之前，我们先来回答一个更简单的问题：到底为什么要用线程?

事实上，使用线程至少有两个主要原因。第一个很简单：**并行**（parallelism）。假设你在写一个对非常大的数组做运算的程序，比如把两个大数组相加，或者给数组中每个元素的值都加上某个数。如果只在单个处理器上运行，这个任务很直接：把每个运算做完就行。然而，如果在有多个处理器的系统上运行这个程序，就有可能让每个处理器各做一部分工作，从而大大加快速度。把普通的**单线程**程序改造成能在多个 CPU 上完成这类工作的程序，这项任务叫作**并行化**（parallelization）；每个 CPU 用一个线程来做这部分工作，是让程序在现代硬件上跑得更快的一种自然而常见的做法。

第二个原因要微妙一些：避免程序因为 I/O 缓慢而停滞不前。假设你在写一个执行各种 I/O 的程序：可能在等待发送或接收消息，可能在等待显式的磁盘 I/O 完成，甚至可能在（隐式地）等待缺页异常（page fault）处理完。与其干等，程序可能更希望做点别的事，比如利用 CPU 做计算，或者再发出更多的 I/O 请求。使用线程是避免卡住的一种自然方式：程序中一个线程在等待时（即阻塞在 I/O 上），CPU 调度程序可以切换到其他已经就绪、能做有用工作的线程。线程让单个程序内部的 I/O 可以和其他活动**重叠**（overlap）进行，就

像**多道程序设计**（multiprogramming）在不同程序的进程之间做到的那样；正因如此，许多现代服务器应用（Web 服务器、数据库管理系统等）在实现中都用到了线程。

当然，上面两种情况也都可以用多个进程来代替线程。不过，线程共享地址空间，因此共享数据很容易，构建这类程序时自然会选择线程。如果是逻辑上相互独立、几乎不需要共享内存中数据结构的任务，进程是更稳妥的选择。

26.2 一个例子：创建线程

我们来看一些细节。假设我们要运行一个程序，它创建两个线程，每个线程各自做一些独立的工作，在这里就是打印“A”或“B”。代码如图 26.2（第 3 页）所示。

主程序创建了两个线程，它们都会运行函数 `mythread()`，只是参数不同（字符串 `A` 或 `B`）。线程一旦创建，就可能立刻开始运行（取决于调度程序的心情）；也可能被置于“就绪”而非“运行”状态，因此暂时还不运行。当然，在多处理器上，这两个线程甚至可能同时运行，不过我们暂时先不考虑这种可能。

创建了这两个线程（我们称之为 T1 和 T2）之后，主线程调用 `pthread_join()`，等待某个特定的线程结束。它调用了两次，从而确保 T1 和 T2 都运行完毕之后，主线程才能再次运行；主线程再次运行时，会打印“main: end”并退出。总的来说，这次运行一共用到了三个线程：主线程、T1 和 T2。

```
1  #include <stdio.h>
2  #include <assert.h>
3  #include <pthread.h>
4  #include "common.h"
5  #include "common_threads.h"
6
7  void *mythread(void *arg) {
8      printf("%s\n", (char *) arg);
9      return NULL;
10 }
11
12 int
13 main(int argc, char *argv[]) {
14     pthread_t p1, p2;
15     int rc;
16     printf("main: begin\n");
17     Pthread_create(&p1, NULL, mythread, "A");
18     Pthread_create(&p2, NULL, mythread, "B");
19     // join 会等待线程结束
20     Pthread_join(p1, NULL);
21     Pthread_join(p2, NULL);
22     printf("main: end\n");
23     return 0;
24 }
```

图 26.2：**简单的线程创建代码（t0.c）**

main	线程 1	线程 2
开始运行		
打印“main: begin”		
创建线程 1		
创建线程 2		
等待 T1		
	运行	
	打印“A”	
	返回	
等待 T2		
		运行
		打印“B”
		返回
打印“main: end”		

图 26.3：**线程轨迹（1）**

我们来看看这个小程序可能的执行顺序。在执行图（图 26.3，第 4 页）中，时间从上往下推进，每一列分别表示某个线程（主线程、线程 1 或线程 2）在什么时候运行。

不过请注意，这种顺序并不是唯一可能的顺序。事实上，给定一个指令序列，可能的顺序有很多，取决于调度程序在某个时刻决定运行哪个线程。例如，线程一旦创建就可能立即运行，这样就会得到图 26.4（第 4 页）所示的执行过程。

如果调度程序决定先运行线程 2，那么即使线程 1 创建得更早，我们甚至可能看到“B”在“A”之前打印出来；没有理由认为先创建的线程就会先运行。图 26.5（第 5 页）展示了最后这种执行顺序，线程 2 抢在线程 1 之前登场亮相。

main	线程 1	线程 2
开始运行		
打印“main: begin”		
创建线程 1		
	运行	
	打印“A”	
	返回	
创建线程 2		
		运行
		打印“B”
		返回
等待 T1		
立即返回；*T1 已经结束*		
等待 T2		
立即返回；*T2 已经结束*		
打印“main: end”		

图 26.4：**线程轨迹（2）**

main	线程 1	线程 2
开始运行		
打印“main: begin”		
创建线程 1		
创建线程 2		
		运行
		打印“B”
		返回
等待 T1		
	运行	
	打印“A”	
	返回	
等待 T2		
立即返回；*T2 已经结束*		
打印“main: end”		

图 26.5：**线程轨迹（3）**

由此可见，创建线程有点像调用函数；不同的是，系统并不是先执行函数、再返回调用者，而是为被调用的例程创建一个新的执行线程，让它独立于调用者运行：它可能在创建函数返回之前就开始运行，也可能要晚得多才运行。接下来运行什么，由操作系统的**调度程序**（scheduler）决定；尽管调度程序很可能实现了某种合理的算法，但我们很难知道在任一时刻会运行什么。

从这个例子你可能还能看出，线程让事情变复杂了：光是判断什么时候运行什么，就已经很难了！就算没有并发，计算机也已经够难懂了。遗憾的是，有了并发，情况只会更糟，而且糟得多。

26.3 为什么会更糟：共享数据

上面这个简单的线程例子很有用，它展示了线程是如何创建的，以及线程如何根据调度程序的决定以不同的顺序运行。但它没有展示线程在访问共享数据时会如何相互影响。

我们来设想一个简单的例子：两个线程都要更新同一个全局共享变量。要研究的代码见图 26.6（第 7 页）。

关于这段代码，有几点说明。首先，正如 Stevens 所建议的 [SR05]，我们把线程的创建和 join 例程包装了一下，出错时直接退出；对这么简单的程序来说，出了错（如果真出了错）至少要能发现，但不必做什么聪明的处理（比如直接退出就行）。因此，`Pthread_create()` 只是调用 `pthread_create()`，并确认返回码为 0；如果不是 0，`Pthread_create()` 就打印一条消息然后退出。

其次，我们没有为两个工作线程写两个不同的函数体，而是只用一段代码，并给线程传一个参数（这里是字符串），让每个线程在自己的消息前面打印不同的字母。

最后也是最重要的，我们现在可以看看每个工作线程要做什么：在循环中给共享变量 `counter` 加一个数，一共加 1000 万次（1e7）。因此，期望的最终结果是 20,000,000。

现在编译并运行这个程序，看看它的表现。有时候，一切都和预期的一样：

```
prompt> gcc -o main main.c -Wall -pthread; ./main
main: begin (counter = 0)
A: begin
B: begin
A: done
B: done
main: done with both (counter = 20000000)
```

遗憾的是，运行这段代码时，即使在单处理器上，也不一定能得到想要的结果。有时会得到：

```
prompt> ./main
main: begin (counter = 0)
A: begin
B: begin
A: done
B: done
main: done with both (counter = 19345221)
```

我们再试一次，看看是不是自己疯了。毕竟，老师不是教过你，计算机应该产生**确定的**（deterministic）结果吗?！难道你的教授们一直在骗你？（倒吸一口凉气）

```
prompt> ./main
main: begin (counter = 0)
A: begin
B: begin
A: done
B: done
main: done with both (counter = 19221041)
```

不仅每次运行的结果都是错的，而且每次的结果还不一样！一个大问题摆在面前：为什么会这样?

26.4 问题的核心：不受控制的调度

要理解为什么会这样，我们必须了解编译器为更新 `counter` 生成的代码序列。在这个例子中，我们只是想给 `counter` 加一个数（1）。因此，完成这件事的代码序列大概是这样的（x86）：

```
mov 0x8049a1c, %eax
add $0x1, %eax
mov %eax, 0x8049a1c
```

这个例子假设变量 `counter` 位于地址 0x8049a1c。在这三条指令中，x86 的 `mov` 指令先取出该地址处的内存值，放进寄存器 `eax`。然后执行 `add`，把 1（0x1）加到 `eax` 寄存器的内容上；最后，把 `eax` 的内容存回内存中的同一地址。

```
#include <stdio.h>
#include <pthread.h>
#include "common.h"
#include "common_threads.h"

static volatile int counter = 0;

// mythread()
//
// 在循环中反复给 counter 加 1
// 当然，正常人不会这样给 counter 加上 10,000,000，
// 但用它来展示问题再好不过。
//
void *mythread(void *arg) {
    printf("%s: begin\n", (char *) arg);
    int i;
    for (i = 0; i < 1e7; i++) {
        counter = counter + 1;
    }
    printf("%s: done\n", (char *) arg);
    return NULL;
}

// main()
//
// 只是启动两个线程（pthread_create），
// 然后等待它们结束（pthread_join）
//
int main(int argc, char *argv[]) {
    pthread_t p1, p2;
    printf("main: begin (counter = %d)\n", counter);
    Pthread_create(&p1, NULL, mythread, "A");
    Pthread_create(&p2, NULL, mythread, "B");

    // join 会等待线程结束
    Pthread_join(p1, NULL);
    Pthread_join(p2, NULL);
    printf("main: done with both (counter = %d)\n",
           counter);
    return 0;
}
```

图 26.6：**共享数据：糟了（t1.c）**

假设两个线程中的一个（线程 1）进入了这段代码，即将把 `counter` 加一。它把 `counter` 的值（假设一开始是 50）加载到自己的寄存器 `eax` 中。于是，线程 1 的 `eax=50`。然后它把寄存器加一，于是 `eax=51`。这时，不幸的事情发生了：时钟中断触发了；于是，

提示：了解并使用你的工具

你应该不断学习新工具，它们能帮你编写、调试和理解计算机系统。这里我们用一个很好用的工具，叫作**反汇编器**（disassembler）。对可执行文件运行反汇编器，它会显示程序由哪些汇编指令组成。例如，如果想了解更新计数器的底层代码（就像我们的例子），可以运行 `objdump`（Linux）来查看汇编代码：

```
prompt> objdump -d main
```

这样会得到程序中所有指令的一份长长的清单，标注得整整齐齐（如果编译时加了 `-g` 选项，清单里还会带上程序的符号信息）。`objdump` 只是你应该学会使用的众多工具之一；调试器 `gdb`、内存分析工具 `valgrind` 或 `purify`，当然还有编译器本身，都值得你花时间深入了解。工具用得越好，你就能构建出越好的系统。

操作系统把当前运行线程的状态（它的 PC、包括 `eax` 在内的寄存器等）保存到这个线程的 TCB 中。

接着，更糟的事情发生了：线程 2 被选中运行，它也进入了同一段代码。它同样执行第一条指令，取出 `counter` 的值，放进它自己的 `eax`（记住：每个线程运行时都有自己私有的寄存器；上下文切换代码会保存和恢复这些寄存器，从而把它们**虚拟化**（virtualized）了）。此时 `counter` 的值仍然是 50，因此线程 2 的 `eax=50`。假设线程 2 接着执行了后面两条指令：把 `eax` 加 1（于是 `eax=51`），然后把 `eax` 的内容存入 `counter`（地址 0x8049a1c）。于是，全局变量 `counter` 现在的值是 51。

最后，又发生了一次上下文切换，线程 1 恢复运行。回想一下，它刚刚执行完 `mov` 和 `add`，现在正要执行最后那条 `mov` 指令。还记得它的 `eax=51`。于是，最后那条 `mov` 指令执行，把这个值存进内存；`counter` 又一次被设成了 51。

简单地说，发生的事情是这样的：给 `counter` 加一的代码运行了两次，可是 `counter` 从 50 开始，现在却只等于 51。如果程序是“正确”的，变量 `counter` 应该等于 52。

为了更好地理解这个问题，我们来看一份详细的执行轨迹。在这个例子中，假设上面的代码加载到内存地址 100 处，排布如下（习惯了整齐的类 RISC 指令集的读者请注意：x86 的指令是变长的；这条 `mov` 指令占 5 个字节，而 `add` 只占 3 个字节）：

```
100 mov    0x8049a1c, %eax
105 add    $0x1, %eax
108 mov    %eax, 0x8049a1c
```

在这些假设下，发生的情况如图 26.7（第 9 页）所示。假设计数器的初始值是 50，请跟着这个例子走一遍，确保你明白发生了什么。

这里演示的情况叫作**竞态条件**（race condition，也叫竞争条件），更具体地说，是**数据竞争**（data race）：结果取决于代码执行的时机。运气不好时（即上下文切换恰好发生在执行过程中不合时宜的地方），就会得到错误的结果。事实上，每次得到的结果都可能不同；因此，我们得到的不是那种漂亮的**确定性**计算（这是我们对计算机的一贯期望），而是一个**不**

			（指令执行后）		
OS	**线程 1**	**线程 2**	**PC**	**eax**	**counter**
	临界区之前		100	0	50
	`mov 8049a1c,%eax`		105	**50**	50
	`add $0x1,%eax`		108	**51**	50
中断					
保存 *T1*					
恢复 *T2*			100	0	50
		`mov 8049a1c,%eax`	105	**50**	50
		`add $0x1,%eax`	108	**51**	50
		`mov %eax,8049a1c`	113	51	**51**
中断					
保存 *T2*					
恢复 *T1*			108	51	51
	`mov %eax,8049a1c`		113	51	**51**

图 26.7：**问题的近距离特写**

确定的（indeterminate）结果：谁也不知道输出会是什么，而且每次运行的输出很可能都不一样。

由于多个线程执行这段代码可能引发竞态条件，我们把这段代码称为**临界区**（critical section）。临界区是访问共享变量（或者更一般地说，共享资源）的一段代码，不能由多个线程同时执行。

对这段代码，我们真正想要的是所谓的**互斥**（mutual exclusion）。这一性质保证：如果一个线程正在临界区内执行，其他线程就无法进入。

顺便说一下，这些术语几乎都是 Edsger Dijkstra 创造的。他是这一领域的先驱，正是因为这项工作以及其他成果获得了图灵奖。关于这个问题，他 1968 年的论文“Cooperating Sequential Processes”[D68] 有一段清晰得惊人的描述，值得一读。在本书的这一部分，我们还会多次听到 Dijkstra 的名字。

26.5 对原子性的期望

解决这个问题的一种办法是提供更强大的指令，一步就完成我们需要做的事，从而杜绝不合时宜的中断。例如，假设我们有一条超级指令，长这样：

```
memory-add 0x8049a1c, $0x1
```

假设这条指令把一个值加到某个内存位置上，并且硬件保证它以原子方式执行；这条指令执行时，会按预期完成更新。它不可能在执行到一半时被中断，因为这正是硬件给我们的保证：中断发生时，这条指令要么根本没有执行，要么已经执行完毕，不存在中间状态。硬件可真美好，不是吗？

在这里，**原子地**（atomically）的意思是“作为一个整体”，有时我们也理解为“全有或全无”。我们希望的是以原子方式执行这三条指令：

提示：使用原子操作
原子操作是构建计算机系统时最强大的底层技术之一，从计算机体系结构，到并发代码（也就是我们这里研究的内容），到文件系统（我们很快就会学到），再到数据库管理系统，甚至分布式系统，都用得到它 [L+93]。
让一系列动作成为**原子的**（atomic），背后的思想用一句话就能说清："要么全做，要么全不做"。在外界看来，要么你想打包在一起的所有动作都发生了，要么一个也没发生，看不到任何中间状态。有时，把许多动作打包成一个原子动作的做法叫作**事务**（transaction），这一思想在数据库和事务处理领域得到了非常深入的发展 [GR92]。
在探讨并发这个主题时，我们会用同步原语把一小段指令序列变成原子的执行块；不过我们会看到，原子性的思想远不止于此。例如，文件系统会使用日志（journaling）或写时复制（copy-on-write）等技术，以原子的方式改变磁盘上的状态，这对系统出故障时仍能正确运行至关重要。如果现在还看不明白，别担心，以后某一章会讲清楚的。

```
mov 0x8049a1c, %eax
add $0x1, %eax
mov %eax, 0x8049a1c
```

正如前面所说，如果有一条指令能完成这件事，我们只需发出这条指令就行了。但一般情况下，我们不会有这样的指令。想象一下，我们正在构建一棵并发的 B 树，并且想要更新它；难道真要让硬件支持一条"原子更新 B 树"的指令吗？大概不会，至少在一个正常的指令集里不会。

所以，我们的做法是向硬件要几条有用的指令，在它们的基础上构建一套通用的**同步原语**（synchronization primitive）。借助这些硬件支持，再加上操作系统的一些帮助，我们就能写出以同步、受控的方式访问临界区的多线程代码，从而在并发执行这种棘手的情况下，依然可靠地得出正确的结果。很棒，对吧？

这就是本书这一部分要研究的问题。这是一个美妙而困难的问题，应该会让你的脑袋（有点）疼。如果没疼，说明你还没懂！继续钻研，直到头疼为止，那时你就知道自己走对了方向。到那个时候，就休息一下吧，我们可不希望你头疼得太厉害。

关键问题：如何支持同步
为了构建有用的同步原语，我们需要硬件提供什么支持？需要操作系统提供什么支持？怎样才能正确、高效地构建这些原语？程序怎样使用它们才能得到想要的结果？

26.6 另一个问题：等待另一个线程

本章在提出并发问题时，好像线程之间只有一种交互：访问共享变量，以及由此需要为临界区提供原子性。事实上，还有另一种常见的交互：一个线程必须等另一个线程完成某个动作，才能继续执行。例如，进程执行磁盘 I/O 时会进入睡眠；I/O 完成后，需要把进程从睡眠中唤醒，让它继续执行。

 © 2008–25, ARPACI-DUSSEAU

补充：并发的关键术语
临界区、竞态条件、不确定性、互斥

这四个术语是并发代码的核心，所以我们觉得值得把它们单独拎出来讲一讲。更多细节请参阅 Dijkstra 的一些早期工作 [D65, D68]。

- **临界区**是访问共享资源的一段代码，共享资源通常是变量或数据结构。
- 如果多个执行线程大致同时进入临界区，都试图更新共享的数据结构，导致出人意料（而且很可能不是我们想要）的结果，就产生了**竞态条件**（或者叫**数据竞争** [NM92]）。
- **不确定的**程序包含一个或多个竞态条件；程序的输出每次运行都不一样，取决于哪些线程在什么时候运行。因此结果不是**确定的**，而确定性恰恰是我们通常对计算机系统的期望。
- 为了避免这些问题，线程应该使用某种**互斥**原语；这样就能保证只有一个线程进入临界区，从而避免竞争，得到确定的程序输出。

因此，在接下来的几章里，我们不仅要研究如何构建同步原语来支持原子性，还要研究支持这种睡眠/唤醒交互的机制，这种交互在多线程程序中很常见。如果现在还不明白，没关系！等你读到**条件变量**（condition variable）那一章，很快就会明白。如果到那时还不明白，那就不太好了，你应该把那一章读一遍（再读一遍），直到弄明白为止。

26.7 小结：为什么要在操作系统课上讲这些？

在结束之前，你可能会问：为什么要在操作系统课上学这些？答案用一个词就能概括：“历史”。操作系统是第一个并发程序，许多技术最初就是为在操作系统内部使用而发明的。后来有了多线程进程，应用程序员也不得不考虑这些问题。

例如，设想有两个进程在运行。假设它们都调用 `write()` 写同一个文件，而且都想把数据追加到文件末尾（即把数据加到文件的最后，使文件变长）。为此，两个进程都必须分配一个新块，在文件的 inode（索引节点）里记下这个块的位置，并修改文件的大小，反映出变大后的长度（还有其他一些工作；本书第三部分会更详细地讲文件）。由于中断随时可能发生，更新这些共享结构的代码（例如用于分配的位图，或者文件的 inode）就是临界区；因此，从引入中断的那一刻起，操作系统的设计者就不得不操心操作系统如何更新内部结构。一次不合时宜的中断，就会引发上面描述的所有问题。毫不奇怪，页表、进程列表、文件系统结构，以及几乎所有内核数据结构，都必须使用恰当的同步原语小心地访问，才能正确工作。

 © 2008–25, ARPACI-DUSSEAU

参考文献

[D65] "Solution of a problem in concurrent programming control" by E. W. Dijkstra. Communications of the ACM, 8(9):569, September 1965. *Pointed to as the first paper of Dijkstra's where he outlines the mutual exclusion problem and a solution. The solution, however, is not widely used; advanced hardware and OS support is needed, as we will see in the coming chapters.*

[D68] "Cooperating sequential processes" by Edsger W. Dijkstra. 1968. Available at this site: http://www.cs.utexas.edu/users/EWD/ewd01xx/EWD123.PDF. *Dijkstra has an amazing number of his old papers, notes, and thoughts recorded (for posterity) on this website at the last place he worked, the University of Texas. Much of his foundational work, however, was done years earlier while he was at the Technische Hogeschool Eindhoven (THE), including this famous paper on "cooperating sequential processes", which basically outlines all of the thinking that has to go into writing multi-threaded programs. Dijkstra discovered much of this while working on an operating system named after his school: the "THE" operating system (said "T", "H", "E", and not like the word "the").*

[GR92] "Transaction Processing: Concepts and Techniques" by Jim Gray and Andreas Reuter. Morgan Kaufmann, September 1992. *This book is the bible of transaction processing, written by one of the legends of the field, Jim Gray. It is, for this reason, also considered Jim Gray's "brain dump", in which he wrote down everything he knows about how database management systems work. Sadly, Gray passed away tragically a few years back, and many of us lost a friend and great mentor, including the co-authors of said book, who were lucky enough to interact with Gray during their graduate school years.*

[L+93] "Atomic Transactions" by Nancy Lynch, Michael Merritt, William Weihl, Alan Fekete. Morgan Kaufmann, August 1993. *A nice text on some of the theory and practice of atomic transactions for distributed systems. Perhaps a bit formal for some, but lots of good material is found herein.*

[NM92] "What Are Race Conditions? Some Issues and Formalizations" by Robert H. B. Netzer and Barton P. Miller. ACM Letters on Programming Languages and Systems, Volume 1:1, March 1992. *An excellent discussion of the different types of races found in concurrent programs. In this chapter (and the next few), we focus on data races, but later we will broaden to discuss* ***general races*** *as well.*

[SR05] "Advanced Programming in the UNIX Environment" by W. Richard Stevens and Stephen A. Rago. Addison-Wesley, 2005. *As we've said many times, buy this book, and read it, in little chunks, preferably before going to bed. This way, you will actually fall asleep more quickly; more importantly, you learn a little more about how to become a serious UNIX programmer.*

 © 2008–25, ARPACI-DUSSEAU

作业（模拟）

程序 x86.py 可以让你看到，不同的线程交错执行方式是如何导致或避免竞态条件的。请先阅读 README，了解这个程序怎么用，然后回答下面的问题。

问题

1. 我们先来看一个简单的程序 loop.s。首先，把它读懂。然后用这些参数运行它：./x86.py -t 1 -p loop.s -i 100 -R dx。这表示只有一个线程，每 100 条指令中断一次，并跟踪寄存器 %dx。运行过程中 %dx 的值会是多少？用 -c 选项检查你的答案：答案显示在左边，是右边那条指令执行之后寄存器（或内存）的值。
2. 同样的代码，换一组选项：./x86.py -p loop.s -t 2 -i 100 -a dx=3,dx=3 -R dx。这表示有两个线程，并把每个线程的 %dx 都初始化为 3。%dx 会出现哪些值？加上 -c 运行来检查。多个线程的存在会影响你的计算吗？这段代码里有竞争吗？
3. 运行这条命令：./x86.py -p loop.s -t 2 -i 3 -r -R dx -a dx=3,dx=3。这会让中断间隔变得很小，而且是随机的；用不同的种子（-s）看看不同的交错情况。中断频率会改变什么吗？
4. 现在换一个程序 looping-race-nolock.s，它访问位于地址 2000 的一个共享变量，我们把这个变量叫作 value。先用单个线程运行它，确认你理解了：./x86.py -p looping-race-nolock.s -t 1 -M 2000。整个运行过程中，value（即内存地址 2000 处的值）是多少？用 -c 检查。
5. 用多次循环、多个线程运行：./x86.py -p looping-race-nolock.s -t 2 -a bx=3 -M 2000。为什么每个线程都循环三次？value 的最终值是多少？
6. 用随机的中断间隔运行：./x86.py -p looping-race-nolock.s -t 2 -M 2000 -i 4 -r -s 0，并换用不同的种子（-s 1、-s 2 等）。只看线程的交错情况，你能判断出 value 的最终值吗？中断的时机重要吗？中断发生在哪里是安全的？哪里不安全？换句话说，临界区究竟在哪里？
7. 现在来看固定的中断间隔：./x86.py -p looping-race-nolock.s -a bx=1 -t 2 -M 2000 -i 1。共享变量 value 的最终值会是多少？改成 -i 2、-i 3 等又会怎样？在哪些中断间隔下，程序能给出“正确”的答案？
8. 用更多次循环运行同样的程序（例如设置 -a bx=100）。哪些中断间隔（-i）能得到正确的结果？哪些间隔的结果出人意料？
9. 最后一个程序：wait-for-me.s。运行 ./x86.py -p wait-for-me.s -a ax=1,ax=0 -R ax -M 2000。这会把线程 0 的 %ax 寄存器设为 1，线程 1 的设

为 0，并监视 `%ax` 和内存位置 2000。这段代码应该表现出怎样的行为？两个线程怎样使用位置 2000 处的值？它的最终值会是多少？

10. 现在把输入对调：`./x86.py -p wait-for-me.s -a ax=0,ax=1 -R ax -M 2000`。这两个线程表现如何？线程 0 在做什么？改变中断间隔（例如 `-i 1000`，或者改用随机间隔）会怎样改变跟踪结果？这个程序是否高效地利用了 CPU？